



UNIVERSIDADE DO VALE DO ITAJAÍ
CIÊNCIA DA COMPUTAÇÃO – PROJETO DE SISTEMAS DIGITAIS
PROF. PhD. DOUGLAS ROSSI DE MELO

GUSTAVO HENRIQUE STAHL MÜLLER

PROJETO DE BLOCOS CONSTRUTIVOS

ITAJAÍ
2022

SUMÁRIO

1. INTRODUÇÃO.....	3
2. DESENVOLVIMENTO.....	4
2.1. DEFINIÇÃO DO PROJETO	4
2.2. SOMADOR COMPLETO DE UM BIT	5
2.3. SOMADOR DE QUATRO BITS.....	8
2.4. BITWISE “AND” DE QUATRO BITS	12
2.5. BITWISE “OR” DE QUATRO BITS	14
2.6. BITWISE “XOR” DE QUATRO BITS.....	16
2.7. INVERSOR DE QUATRO BITS.....	19
2.8. INCREMENTADOR DE QUATRO BITS	21
2.9. SUBTRATOR DE QUATRO BITS.....	23
2.10. REGISTRADOR DE QUATRO BITS	27
3. PROJETO FINAL DA ULA.....	30
3.1. PROTOTIPAÇÃO EM FPGA	37
3.1.1. MAPEAMENTO DOS PINOS.....	38
3.1.2. TESTES NO FPGA	38

1. INTRODUÇÃO

Esse relatório detalha o processo de criação de uma ULA (unidade lógica e aritmética) com as ferramentas VHDL, Quartus Prime e Questa. A ULA foi desenvolvida com o objetivo de consolidar o aprendizado sobre projeto de blocos construtivos, e foi posteriormente prototipada em um FPGA Cyclone II da Altera.

2. DESENVOLVIMENTO

2.1. DEFINIÇÃO DO PROJETO

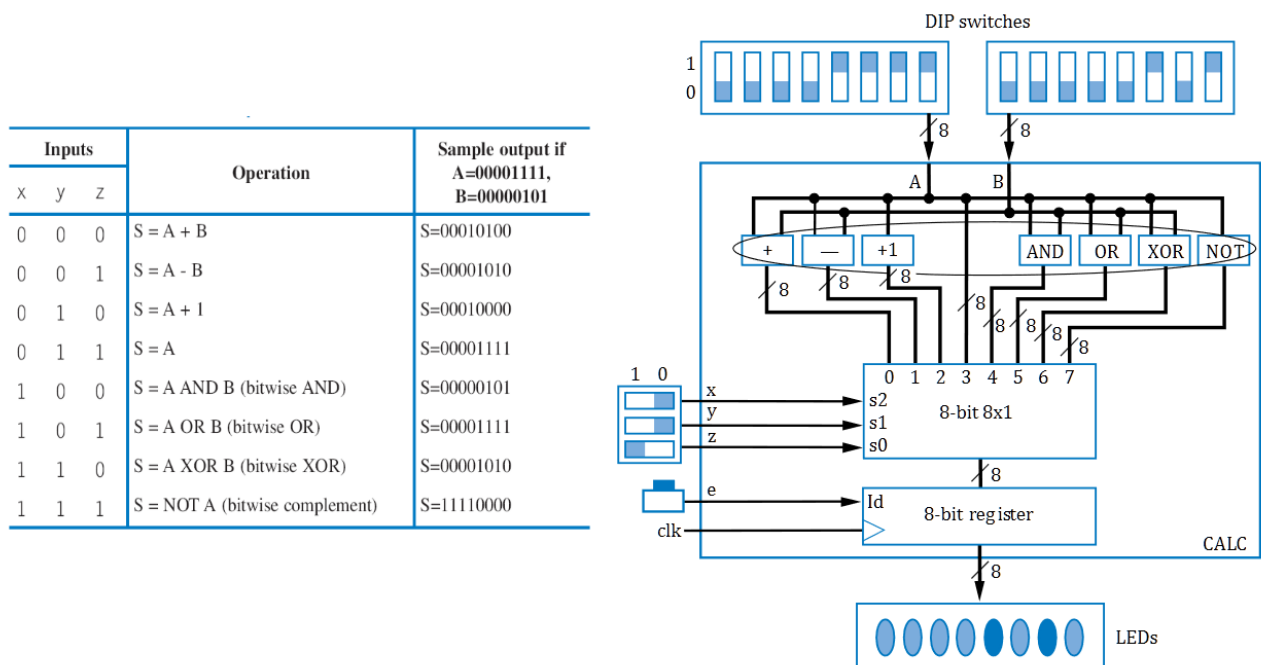
A unidade lógica e aritmética que será descrita, sintetizada e simulada nesse trabalho contém cinco entradas e uma saída, sendo elas:

- Duas entradas de dados de quatro bits cada (A e B);
- Uma entrada de três bits que seleciona a operação que será realizada (SEL);
- Uma entrada que habilita o salvamento do resultado da operação em um registrador de quatro bits (LOAD);
- Uma entrada de relógio que sincroniza o registrador (CLK);

A ULA deve ser capaz de realizar oito operações diferentes, com base no valor do seletor. A tabela verdade e um diagrama esquemático são mostrados na figura 1.

1. $A + B$, quando o seletor é igual à “000”;
2. $A - B$, quando o seletor é igual à “001”;
3. $A + 1$, quando o seletor é igual à “010”;
4. A , quando o seletor é igual à “011”;
5. $A \text{ AND } B$, quando o seletor é igual à “100”;
6. $A \text{ OR } B$, quando o seletor é igual à “101”;
7. $A \text{ XOR } B$, quando o seletor é igual à “110”;
8. $\text{NOT } A$, quando o seletor é igual à “111”;

Figura 1 – Tabela verdade e diagrama esquemático da ULA.



Fonte: O autor.

Como requisitado pelo trabalho, a construção das funções da ULA foi realizada pela conexão entre blocos menores.

2.2. SOMADOR COMPLETO DE UM BIT

O somador completo de um bit é responsável por somar dois bits de dados com um *carry in*, e produzir um bit de resultado e um *carry out*. Esse circuito é reutilizado quatro vezes de maneira encadeada no somador de quatro bits, e o somador de quatro bits em si é reusado em outros componentes, como o subtrator de quatro bits e o incrementador de quatro bits.

Figura 2 – Tabela verdade do somador completo de 1 bit.

A	B	CARRY IN	R	CARRY OUT
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Fonte: O autor.

Figura 3 – Entidade e arquitetura do somador completo de 1 bit.

```
library IEEE;
use IEEE.std_logic_1164.all;

entity full_adder_1bit is
    port
    (
        i_A: in std_logic;
        i_B: in std_logic;
        i_CARRY_IN: in std_logic;
        o_R: out std_logic;
        o_CARRY_OUT: out std_logic
    );
end full_adder_1bit;

architecture arch_full_adder_1bit of full_adder_1bit is
begin
    o_R <= i_A XOR i_B XOR i_CARRY_IN;
```

```

        o_CARRY_OUT <= (i_A AND i_B) OR (i_A AND i_CARRY_IN) OR (i_B
AND i_CARRY_IN);
end arch_full_adder_1bit;

```

Fonte: O autor.

Figura 4 – Testbench do somador completo de 1 bit.

```

library IEEE;
use IEEE.std_logic_1164.all;

entity full_adder_1bit_tb is
end full_adder_1bit_tb;

architecture arch_full_adder_1bit_tb of full_adder_1bit_tb is

component full_adder_1bit is
    port
    (
        i_A: in std_logic;
        i_B: in std_logic;
        i_CARRY_IN: in std_logic;
        o_R: out std_logic;
        o_CARRY_OUT: out std_logic
    );
end component;

signal w_A: std_logic;
signal w_B: std_logic;
signal w_CARRY_IN: std_logic;
signal w_R: std_logic;
signal w_CARRY_OUT: std_logic;

begin

    u_FULL_ADDER: full_adder_1bit port map (w_A, w_B, w_CARRY_IN, w_R, w_CARRY_OUT);

    process
    begin
        -- Test 1
        w_A <= '0';
        w_B <= '0';
        w_CARRY_IN <= '0';

        wait for 1ns;
        assert (w_R = '0') report "Fail full_adder_1bit -> Test 1" severity error;
        assert (w_CARRY_OUT = '0') report "Fail full_adder_1bit -> Test 1" severity error;

        -- Test 2
        w_A <= '0';
        w_B <= '0';
        w_CARRY_IN <= '1';

        wait for 1ns;
        assert (w_R = '1') report "Fail full_adder_1bit -> Test 2" severity error;
        assert (w_CARRY_OUT = '0') report "Fail full_adder_1bit -> Test 2" severity error;

        -- Test 3
        w_A <= '0';
        w_B <= '1';
        w_CARRY_IN <= '0';

        wait for 1ns;
        assert (w_R = '1') report "Fail full_adder_1bit -> Test 3" severity error;
        assert (w_CARRY_OUT = '0') report "Fail full_adder_1bit -> Test 3" severity error;
    end process;
end arch_full_adder_1bit_tb;

```

```

-- Test 4
w_A <= '0';
w_B <= '1';
w_CARRY_IN <= '1';

wait for 1ns;
assert (w_R = '0') report "Fail full_adder_1bit -> Test 4" severity error;
assert (w_CARRY_OUT = '1') report "Fail full_adder_1bit -> Test 4" severity error;

-- Test 5
w_A <= '1';
w_B <= '0';
w_CARRY_IN <= '0';

wait for 1ns;
assert (w_R = '1') report "Fail full_adder_1bit -> Test 5" severity error;
assert (w_CARRY_OUT = '0') report "Fail full_adder_1bit -> Test 5" severity error;

-- Test 6
w_A <= '1';
w_B <= '0';
w_CARRY_IN <= '1';

wait for 1ns;
assert (w_R = '0') report "Fail full_adder_1bit -> Test 6" severity error;
assert (w_CARRY_OUT = '1') report "Fail full_adder_1bit -> Test 6" severity error;

-- Test 7
w_A <= '1';
w_B <= '1';
w_CARRY_IN <= '0';

wait for 1ns;
assert (w_R = '0') report "Fail full_adder_1bit -> Test 7" severity error;
assert (w_CARRY_OUT = '1') report "Fail full_adder_1bit -> Test 7" severity error;

-- Test 8
w_A <= '1';
w_B <= '1';
w_CARRY_IN <= '1';

wait for 1ns;
assert (w_R = '1') report "Fail full_adder_1bit -> Test 8" severity error;
assert (w_CARRY_OUT = '1') report "Fail full_adder_1bit -> Test 8" severity error;

wait;
end process;
end arch_full_adder_1bit_tb;

```

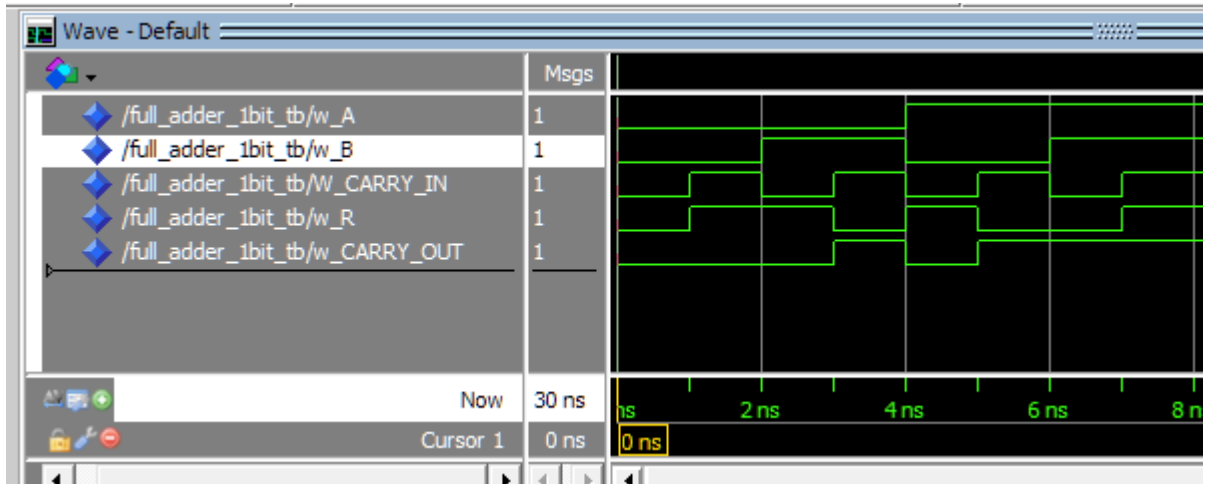
Fonte: O autor.

Através do diagrama de forma de onda é possível comparar os resultados com a tabela verdade e identificar que o circuito gera as saídas corretas em todas as combinações possíveis de valores de entrada:

- Quando nenhuma entrada é 1 (instante 0ns), R e CARRY_OUT são iguais a 0 (simbolizando o número 0);
- Quando uma das entradas é 1 (instantes 1ns, 2ns e 4ns), R é 1 e CARRY_OUT é 0 (simbolizando o número 1);

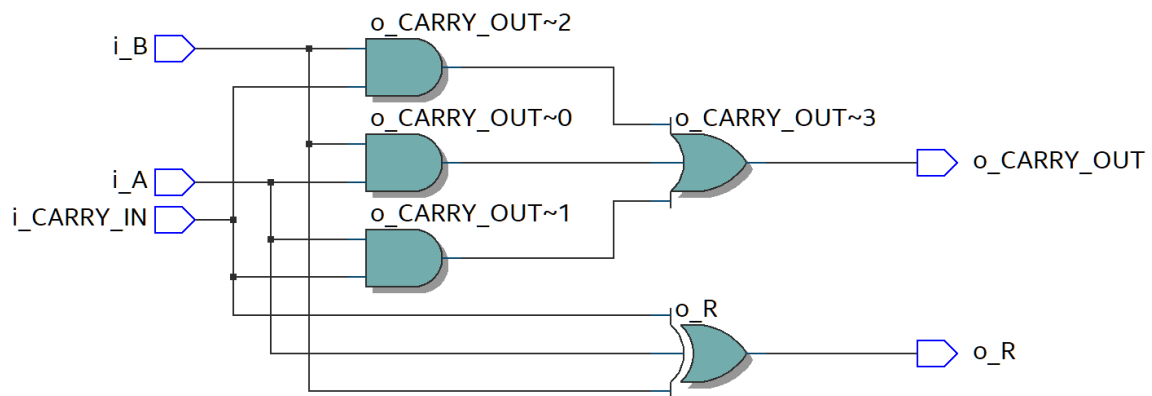
- Quando duas das entradas é 1 (instantes 3ns, 5ns e 6ns) R é 0 e CARRY_OUT é 1 (simbolizando o número 2);
- Quando as três entradas são 1 (instante 7ns), R é 1 e CARRY_OUT é 1 (simbolizando o número 3);

Figura 5 – Diagrama de forma de onda da simulação do somador completo de 1 bit.



Fonte: O autor.

Figura 6 – Diagrama RTL do somador completo de um bit.



Fonte: O autor.

2.3. SOMADOR DE QUATRO BITS

O somador de quatro bits soma dois valores de quatro bits e tem como saídas um resultado de quatro bits e um bit de *carry out*. Ele foi implementado pela conexão de quatro somadores completos de um bit, onde o *carry out* do somador passado é o *carry in* do somador seguinte.

Figura 7 – Entidade e arquitetura do somador de quatro bits.

```

library IEEE;
use IEEE.std_logic_1164.all;

entity adder_4bit is
    port
    (
        i_A: in std_logic_vector (3 downto 0);
        i_B: in std_logic_vector (3 downto 0);
        o_R: out std_logic_vector (3 downto 0);
        o_CARRY_OUT: out std_logic
    );
end adder_4bit;

architecture arch_adder_4bit of adder_4bit is

    component full_adder_1bit is
        port
        (
            i_A: in std_logic;
            i_B: in std_logic;
            i_CARRY_IN: in std_logic;
            o_R: out std_logic;
            o_CARRY_OUT: out std_logic
        );
    end component;

    signal w_CARRY_OUT_0: std_logic;
    signal w_CARRY_OUT_1: std_logic;
    signal w_CARRY_OUT_2: std_logic;

    begin
        u_FULL_ADDER_0: full_adder_1bit port map(i_A(0), i_B(0), '0',
        o_R(0), w_CARRY_OUT_0);
        u_FULL_ADDER_1: full_adder_1bit port map(i_A(1), i_B(1),
        w_CARRY_OUT_0, o_R(1), w_CARRY_OUT_2);
        u_FULL_ADDER_2: full_adder_1bit port map(i_A(2), i_B(2),
        w_CARRY_OUT_1, o_R(2), w_CARRY_OUT_1);
        u_FULL_ADDER_3: full_adder_1bit port map(i_A(3), i_B(3),
        w_CARRY_OUT_2, o_R(3), o_CARRY_OUT);
    end arch_adder_4bit;

```

Fonte: O autor.

Figura 8 – Testbench do somador de quatro bits.

```

library IEEE;
use IEEE.std_logic_1164.all;

entity adder_4bit_tb is
end adder_4bit_tb;

architecture arch_adder_4bit_tb of adder_4bit_tb is

component adder_4bit is
    port
    (
        i_A: in std_logic_vector (3 downto 0);
        i_B: in std_logic_vector (3 downto 0);
        o_R: out std_logic_vector (3 downto 0);
        o_CARRY_OUT: out std_logic
    );
end component;

signal w_A: std_logic_vector (3 downto 0);
signal w_B: std_logic_vector (3 downto 0);
signal w_R: std_logic_vector (3 downto 0);
signal w_CARRY_OUT: std_logic;

begin
    u_ADDER: adder_4bit port map (w_A, w_B, w_R, w_CARRY_OUT);

    process
    begin
        -- Test 1
        w_A <= "1001";
        w_B <= "0011";

        wait for 1ns;
        assert (w_R = "1100") report "Fail adder_4bit -> Test 1" severity error;
        assert (w_CARRY_OUT = '0') report "Fail adder_4bit -> Test 1" severity error;

        -- Test 2
        w_A <= "1111";
        w_B <= "0001";

        wait for 1ns;
        assert (w_R = "0000") report "Fail adder_4bit -> Test 2" severity error;
        assert (w_CARRY_OUT = '1') report "Fail adder_4bit -> Test 2" severity error;

        -- Test 3
        w_A <= "0101";
        w_B <= "0010";

        wait for 1ns;
        assert (w_R = "1000") report "Fail adder_4bit -> Test 3" severity error;
        assert (w_CARRY_OUT = '0') report "Fail adder_4bit -> Test 3" severity error;

        wait;
    end process;
end arch_adder_4bit_tb;

wait for 1ns;
assert (w_R = '0') report "Fail full_adder_1bit -> Test 7" severity error;
assert (w_CARRY_OUT = '1') report "Fail full_adder_1bit -> Test 7" severity error;

-- Test 8
w_A <= '1';
w_B <= '1';
w_CARRY_IN <= '1';

```

```

wait for 1ns;
assert (w_R = '1') report "Fail full_adder_1bit -> Test 8" severity error;
assert (w_CARRY_OUT = '1') report "Fail full_adder_1bit -> Test 8" severity error;

wait;
end process;
end arch_full_adder_1bit_tb;

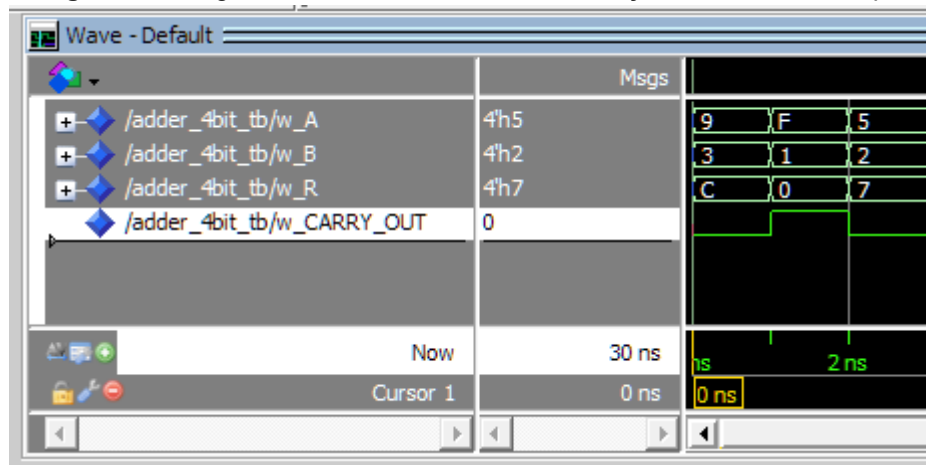
```

Fonte: O autor.

Através do diagrama de forma de onda é possível confirmar que o circuito se comporta da maneira esperada:

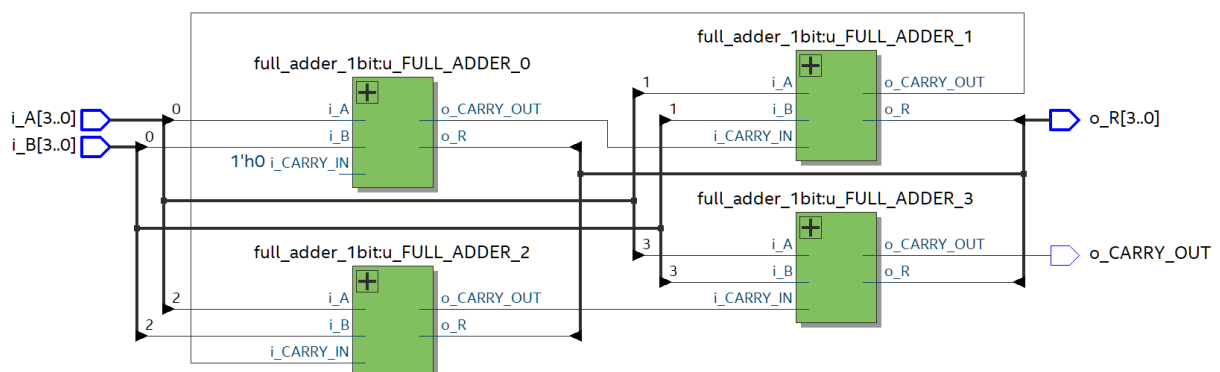
- No instante 0ns, os valores 9 e 3 são somados, produzindo C (12 em hexadecimal) como resultado e 0 como carry out;
- No instante 1ns, os valores F (15 em hexadecimal) e 1 são somados, produzindo 0 como resultado e 1 como carry out, que simboliza o overflow ocorrido;
- No instante 2ns, os valores 5 e 2 são somados, produzindo 7 como resultado e 0 como carry out.

Figura 9 – Diagrama de forma de onda da simulação do somador de quatro bits.



Fonte: O autor.

Figura 10 – Diagrama RTL do somador de quatro bits.



Fonte: O autor.

2.4. BITWISE “AND” DE QUATRO BITS

O circuito *bitwise* AND de quatro bits tem como entrada dois valores de quatro bits e como saída um valor de quatro bits onde todos os seus bits são o resultado da operação AND dos bits correspondentes (de mesmo índice) das entradas.

Figura 11 – Entidade e arquitetura do circuito bitwise AND de quatro bits.

```
library IEEE;
use IEEE.std_logic_1164.all;

entity and_4bit is
    port
    (
        i_A: in std_logic_vector (3 downto 0);
        i_B: in std_logic_vector (3 downto 0);
        o_R: out std_logic_vector (3 downto 0)
    );
end and_4bit;

architecture arch_and_4bit of and_4bit is
begin
    o_R(0) <= i_A(0) AND i_B(0);
    o_R(1) <= i_A(1) AND i_B(1);
    o_R(2) <= i_A(2) AND i_B(2);
    o_R(3) <= i_A(3) AND i_B(3);
end arch_and_4bit;
```

Fonte: O autor.

Figura 12 – Testbench do circuito bitwise AND de quatro bits.

```
library IEEE;
use IEEE.std_logic_1164.all;

entity and_4bit_tb is
end and_4bit_tb;

architecture arch_and_4bit_tb of and_4bit_tb is

    component and_4bit is
        port
        (
            i_A: in std_logic_vector (3 downto 0);
            i_B: in std_logic_vector (3 downto 0);
            o_R: out std_logic_vector (3 downto 0)
        );
    end component;

    signal w_A: std_logic_vector (3 downto 0);
    signal w_B: std_logic_vector (3 downto 0);
    signal w_R: std_logic_vector (3 downto 0);

begin
```

```

u_OR: and_4bit port map (w_A, w_B, w_R);

process
begin
    -- Test 1
    w_A <= "1001";
    w_B <= "1001";

    wait for 1ns;
    assert (w_R = "1001") report "Fail and_4bit -> Test 1" severity error;

    -- Test 2
    w_A <= "0101";
    w_B <= "1110";

    wait for 1ns;
    assert (w_R = "0100") report "Fail and_4bit -> Test 2" severity error;

    -- Test 3
    w_A <= "1010";
    w_B <= "0101";

    wait for 1ns;
    assert (w_R = "0000") report "Fail and_4bit -> Test 3" severity error;

    wait;
end process;
end arch_and_4bit_tb;

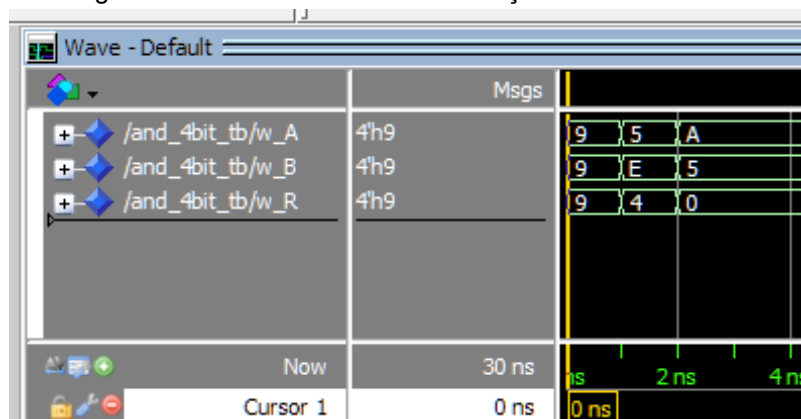
```

Fonte: O autor.

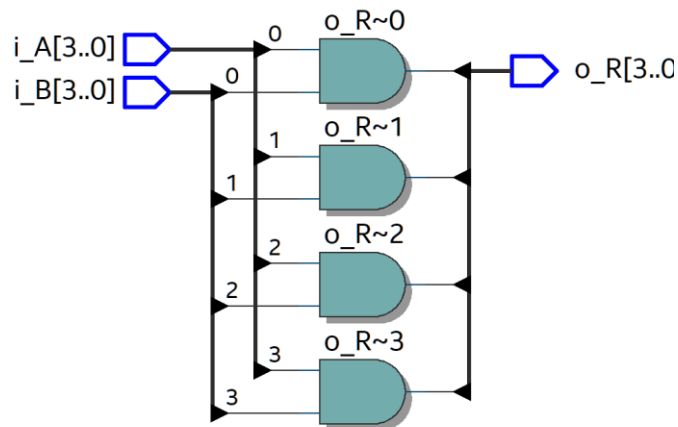
Através do diagrama de forma de onda é possível confirmar que o circuito se comporta da maneira esperada:

- No instante 0ns, dois valores iguais de entrada são testados, que resultam numa saída igual;
- No instante 1ns, os valores 5 (0101) e E (1110) são testados, produzindo o valor 4 (0100);
- No instante 2ns, os valores complementares A (1010) e 5 (0101) são testados, produzindo o valor 0 (0000);

Figura 13 – Diagrama de forma de onda da simulação do circuito bitwise AND de quatro bits.



Fonte: O autor.

Figura 14 – Diagrama RTL do circuito bitwise AND de quatro bits.

Fonte: O autor.

2.5. BITWISE “OR” DE QUATRO BITS

O circuito *bitwise* OR de quatro bits tem como entrada dois valores de quatro bits e como saída um valor de quatro bits onde todos os seus bits são o resultado da operação OR dos bits correspondentes (de mesmo índice) das entradas.

Figura 15 – Entidade e arquitetura do circuito bitwise OR de quatro bits.

```

library IEEE;
use IEEE.std_logic_1164.all;

entity or_4bit is
    port
    (
        i_A: in std_logic_vector (3 downto 0);
        i_B: in std_logic_vector (3 downto 0);
        o_R: out std_logic_vector (3 downto 0)
    );
end or_4bit;

architecture arch_or_4bit of or_4bit is
begin
    o_R(0) <= i_A(0) OR i_B(0);
    o_R(1) <= i_A(1) OR i_B(1);
    o_R(2) <= i_A(2) OR i_B(2);
    o_R(3) <= i_A(3) OR i_B(3);
end arch_or_4bit;

```

Fonte: O autor.

Figura 16 – Testbench do circuito bitwise OR de quatro bits.

```

library IEEE;
use IEEE.std_logic_1164.all;

entity or_4bit_tb is
end or_4bit_tb;

architecture arch_or_4bit_tb of or_4bit_tb is

component or_4bit is
    port
    (
        i_A: in std_logic_vector (3 downto 0);
        i_B: in std_logic_vector (3 downto 0);
        o_R: out std_logic_vector (3 downto 0)
    );
end component;

signal w_A: std_logic_vector (3 downto 0);
signal w_B: std_logic_vector (3 downto 0);
signal w_R: std_logic_vector (3 downto 0);

begin

    u_OR: or_4bit port map (w_A, w_B, w_R);

    process
    begin
        -- Test 1
        w_A <= "1001";
        w_B <= "1001";

        wait for 1ns;
        assert (w_R = "1001") report "Fail or_4bit -> Test 1" severity error;

        -- Test 2
        w_A <= "0101";
        w_B <= "1100";

        wait for 1ns;
        assert (w_R = "1101") report "Fail or_4bit -> Test 2" severity error;

        -- Test 3
        w_A <= "1010";
        w_B <= "0101";

        wait for 1ns;
        assert (w_R = "1111") report "Fail or_4bit -> Test 3" severity error;

        wait;
    end process;
end arch_or_4bit_tb;

```

Fonte: O autor.

Através do diagrama de forma de onda é possível confirmar que o circuito se comporta da maneira esperada:

- No instante 0ns, dois valores iguais de entrada são testados, que resultam numa saída igual;
- No instante 1ns, os valores 5 (0101) e C (1100) são testados, produzindo o valor D (1101);

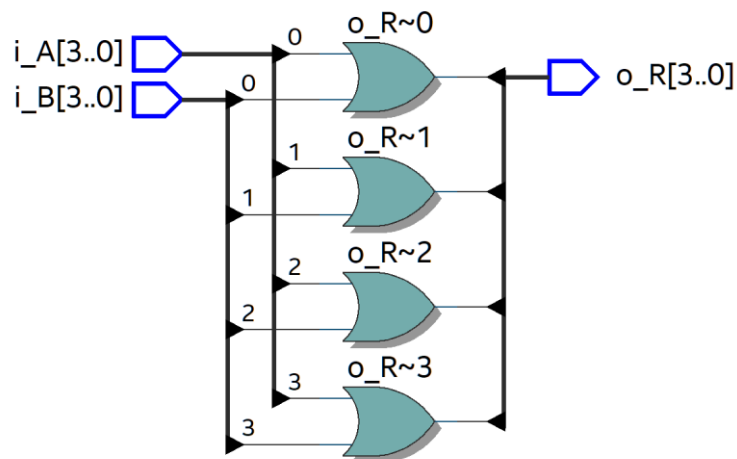
- No instante 2ns, os valores complementares A (1010) e 5 (0101) são testados, produzindo o valor F (1111);

Figura 17 – Diagrama de forma de onda da simulação do circuito bitwise OR de quatro bits.



Fonte: O autor.

Figura 18 – Diagrama RTL do circuito bitwise OR de quatro bits.



Fonte: O autor.

2.6. BITWISE “XOR” DE QUATRO BITS

O circuito *bitwise* XOR de quatro bits tem como entrada dois valores de quatro bits e como saída um valor de quatro bits onde todos os seus bits são o resultado da operação XOR dos bits correspondentes (de mesmo índice) das entradas.

Figura 19 – Entidade e arquitetura do circuito bitwise XOR de quatro bits.

```

library IEEE;
use IEEE.std_logic_1164.all;

entity xor_4bit is
  port
  (

```



```

        i_A: in std_logic_vector (3 downto 0);
        i_B: in std_logic_vector (3 downto 0);
        o_R: out std_logic_vector (3 downto 0)
    );
end xor_4bit;

architecture arch_xor_4bit of xor_4bit is
begin
    o_R(0) <= i_A(0) XOR i_B(0);
    o_R(1) <= i_A(1) XOR i_B(1);
    o_R(2) <= i_A(2) XOR i_B(2);
    o_R(3) <= i_A(3) XOR i_B(3);
end arch_xor_4bit;

```

Fonte: O autor.

Figura 20 – Testbench do circuito bitwise XOR de quatro bits.

```

library IEEE;
use IEEE.std_logic_1164.all;

entity xor_4bit_tb is
end xor_4bit_tb;

architecture arch_xor_4bit_tb of xor_4bit_tb is

    component xor_4bit is
        port
        (
            i_A: in std_logic_vector (3 downto 0);
            i_B: in std_logic_vector (3 downto 0);
            o_R: out std_logic_vector (3 downto 0)
        );
    end component;

    signal w_A: std_logic_vector (3 downto 0);
    signal w_B: std_logic_vector (3 downto 0);
    signal w_R: std_logic_vector (3 downto 0);

begin
    u_XOR: xor_4bit port map (w_A, w_B, w_R);

    process
    begin
        -- Test 1
        w_A <= "1111";
        w_B <= "0000";

        wait for 1ns;
        assert (w_R = "1111") report "Fail xor_4bit -> Test 1" severity error;

        -- Test 2
        w_A <= "0101";
        w_B <= "1100";

        wait for 1ns;
        assert (w_R = "1001") report "Fail xor_4bit -> Test 2" severity error;
    end process;
end arch_xor_4bit_tb;

```

```

-- Test 3
w_A <= "1100";
w_B <= "1111";

wait for 1ns;
assert (w_R = "0011") report "Fail xor_4bit -> Test 3" severity error;

wait;
end process;
end arch_xor_4bit_tb;

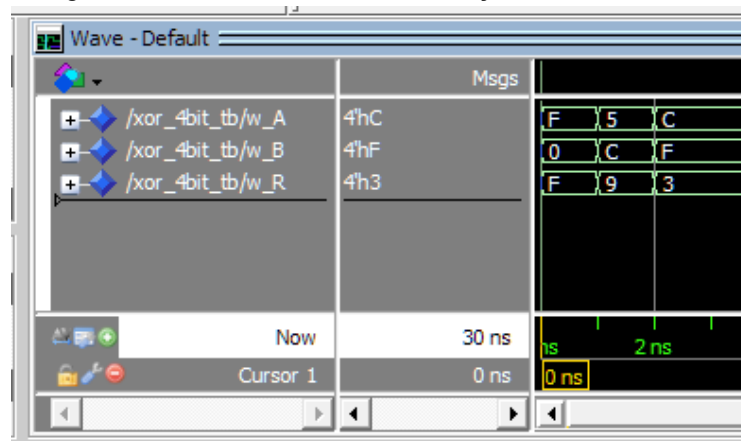
```

Fonte: O autor.

Através do diagrama de forma de onda é possível confirmar que o circuito se comporta da maneira esperada:

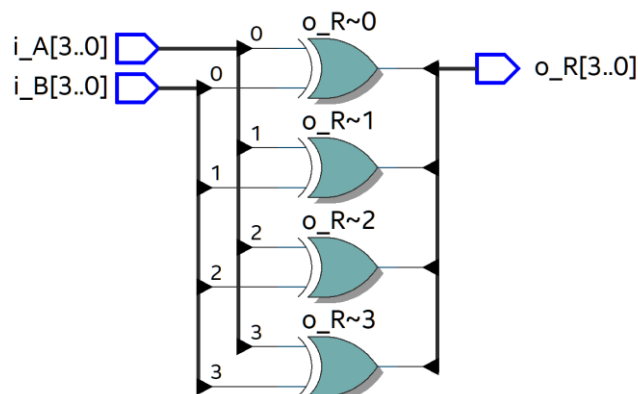
- No instante 0ns, os valores complementares F (1111) e 0 (0000) são testados, produzindo o valor F (1111);
- No instante 1ns, os valores 5 (0101) e C (1100) são testados, produzindo o valor 9 (1001);
- No instante 2ns, os valores complementares C (1100) e F (1111) são testados, produzindo o valor 3 (0011);

Figura 21 – Diagrama de forma de onda da simulação do circuito bitwise XOR de quatro bits.



Fonte: O autor.

Figura 22 – Diagrama RTL do circuito bitwise XOR de quatro bits.



Fonte: O autor.

2.7. INVERSOR DE QUATRO BITS

O inversor de quatro bits tem como entrada um valor de quatro bits e como saída um valor de quatro bits onde todos os seus bits são o resultado da operação NOT dos bits correspondentes (de mesmo índice) da entrada.

Figura 23 – Entidade e arquitetura do inversor de quatro bits.

```
library IEEE;
use IEEE.std_logic_1164.all;

entity inverter_4bit is
    port
    (
        i_A: in std_logic_vector (3 downto 0);
        o_R: out std_logic_vector (3 downto 0)
    );
end inverter_4bit;

architecture arch_inverter_4bit of inverter_4bit is
begin
    o_R(0) <= NOT i_A(0);
    o_R(1) <= NOT i_A(1);
    o_R(2) <= NOT i_A(2);
    o_R(3) <= NOT i_A(3);
end arch_inverter_4bit;
```

Fonte: O autor.

Figura 24 – Testbench do inversor de quatro bits.

```
library IEEE;
use IEEE.std_logic_1164.all;

entity inverter_4bit_tb is
end inverter_4bit_tb;

architecture arch_inverter_4bit_tb of inverter_4bit_tb is

    component inverter_4bit is
        port
        (
            i_A: in std_logic_vector (3 downto 0);
            o_R: out std_logic_vector (3 downto 0)
        );
    end component;

    signal w_A: std_logic_vector (3 downto 0);
    signal w_R: std_logic_vector (3 downto 0);

begin
    u_AND: inverter_4bit port map (w_A, w_R);

    process
```

```

begin
    -- Test 1
    w_A <= "1111";

    wait for 1ns;
    assert (w_R = "0000") report "Fail inverter_4bit -> Test 1" severity error;

    -- Test 2
    w_A <= "0101";

    wait for 1ns;
    assert (w_R = "1010") report "Fail inverter_4bit -> Test 2" severity error;

    -- Test 3
    w_A <= "1100";

    wait for 1ns;
    assert (w_R = "0011") report "Fail inverter_4bit -> Test 3" severity error;

    wait;
end process;
end arch_inverter_4bit_tb;

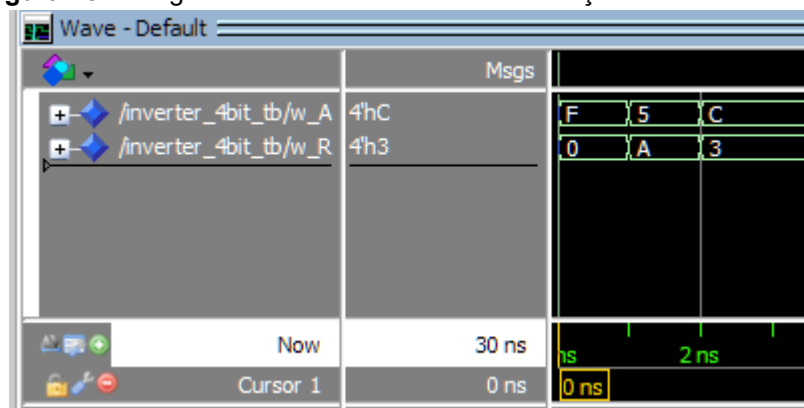
```

Fonte: O autor.

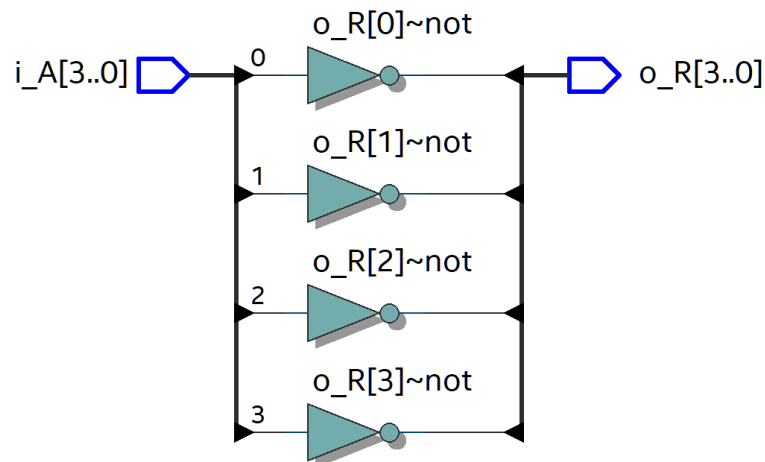
Através do diagrama de forma de onda é possível confirmar que o circuito se comporta da maneira esperada:

- No instante 0ns, o valor F (1111) é testado, produzindo o valor 0 (0000);
- No instante 0ns, o valor 5 (0101) é testado, produzindo o valor A (1010);
- No instante 0ns, o valor C (1100) é testado, produzindo o valor 3 (0011);

Figura 25 – Diagrama de forma de onda da simulação do inversor de quatro bits.



Fonte: O autor.

Figura 26 – Diagrama RTL do inversor de quatro bits.

Fonte: O autor.

2.8. INCREMENTADOR DE QUATRO BITS

O incrementador tem como entrada um valor de quatro bits e como saída um valor de quatro bits e um bit de *carry out*. O incrementador simplesmente acrescenta o valor 1 à entrada usando um somador de quatro bits.

Esse circuito é reutilizado no circuito subtrator, onde será necessário calcular o complemento de dois da segunda entrada.

Figura 27 – Entidade e arquitetura do incrementador de quatro bits.

```
library IEEE;
use IEEE.std_logic_1164.all;

entity incrementer_4bit is
    port
    (
        i_A: in std_logic_vector (3 downto 0);
        o_R: out std_logic_vector (3 downto 0);
        o_CARRY_OUT: out std_logic
    );
end incrementer_4bit;

architecture arch_incrementer_4bit of incrementer_4bit is

    component adder_4bit is
        port
        (
            i_A: in std_logic_vector (3 downto 0);
            i_B: in std_logic_vector (3 downto 0);
```

```

        o_R: out std_logic_vector (3 downto 0);
        o_CARRY_OUT: out std_logic
    );
end component;

begin
    u_ADDER: adder_4bit port map(i_A, "0001", o_R, o_CARRY_OUT);
end arch_incrementer_4bit;

```

Fonte: O autor.

Figura 28 – Testbench do incrementador de quatro bits.

```

library IEEE;
use IEEE.std_logic_1164.all;

entity incrementer_4bit_tb is
end incrementer_4bit_tb;

architecture arch_incrementer_4bit_tb of incrementer_4bit_tb is

component incrementer_4bit is
    port
    (
        i_A: in std_logic_vector (3 downto 0);
        o_R: out std_logic_vector (3 downto 0);
        o_CARRY_OUT: out std_logic
    );
end component;

signal w_A: std_logic_vector (3 downto 0);
signal w_R: std_logic_vector (3 downto 0);
signal w_CARRY_OUT: std_logic;

begin
    u_INCREMENTER: incrementer_4bit port map (w_A, w_R, w_CARRY_OUT);

    process
    begin
        -- Test 1
        w_A <= "1110";

        wait for 1ns;
        assert (w_R = "1111") report "Fail incrementer_4bit -> Test 1" severity error;
        assert (w_CARRY_OUT = '0') report "Fail incrementer_4bit -> Test 1" severity error;

        -- Test 2
        w_A <= "1111";

        wait for 1ns;
        assert (w_R = "0000") report "Fail incrementer_4bit -> Test 2" severity error;
        assert (w_CARRY_OUT = '1') report "Fail incrementer_4bit -> Test 2" severity error;

        -- Test 3
        w_A <= "0101";

        wait for 1ns;
        assert (w_R = "0110") report "Fail incrementer_4bit -> Test 3" severity error;
        assert (w_CARRY_OUT = '0') report "Fail incrementer_4bit -> Test 3" severity error;

        wait;
    end process;
end arch_incrementer_4bit_tb;

```

```

end process;
end arch_incrementer_4bit_tb;

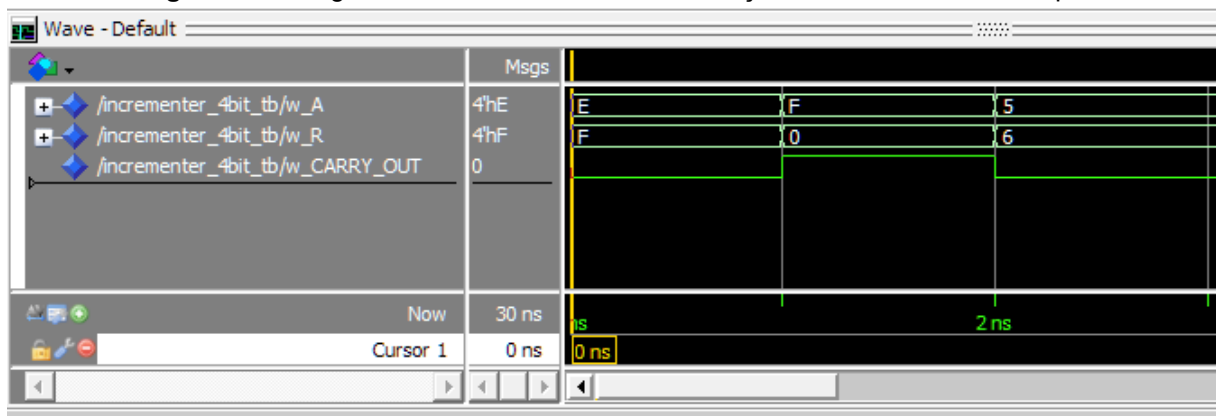
```

Fonte: O autor.

Através do diagrama de forma de onda é possível confirmar que o circuito se comporta da maneira esperada:

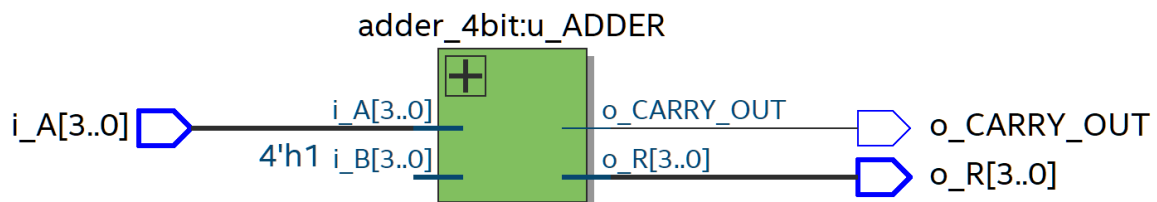
- No instante 0ns, o valor E (1110) é testado, produzindo o valor F (1111) como resultado e o valor 0 como *carry out*;
- No instante 0ns, o valor F (1111) é testado, produzindo o valor 0 (0000) como resultado e o valor 1 como *carry out*;
- No instante 0ns, o valor 5 (0101) é testado, produzindo o valor 6 (0110) como resultado e o valor 0 como *carry out*;

Figura 29 – Diagrama de forma de onda da simulação do incrementador de quatro bits.



Fonte: O autor.

Figura 30 – Diagrama RTL do incrementador de quatro bits.



Fonte: O autor.

2.9. SUBTRATOR DE QUATRO BITS

O subtrator de quatro bits tem como entrada dois valores de quatro bits e como saída um valor de quatro bits que representa o resultado da subtração das entradas e um bit de *carry out*.

Como o sistema de complemento de dois foi usado para representar os números negativos, é possível implementar esse bloco com um somador comum, basta realizar o complemento de dois da segunda entrada antes da soma.

Para calcular o complemento de dois da segunda entrada, é necessário inverter todos os seus bits e somar um. Logo, internamente, o circuito subtrator usa um somador, um inversor e um incrementador.

Figura 31 – Entidade e arquitetura do subtrator de quatro bits.

```
library IEEE;
use IEEE.std_logic_1164.all;

entity subtractor_4bit is
    port
    (
        i_A: in std_logic_vector (3 downto 0);
        i_B: in std_logic_vector (3 downto 0);
        o_R: out std_logic_vector (3 downto 0);
        o_CARRY_OUT: out std_logic
    );
end subtractor_4bit;

architecture arch_subtractor_4bit of subtractor_4bit is

    component inverter_4bit is
        port
        (
            i_A: in std_logic_vector (3 downto 0);
            o_R: out std_logic_vector (3 downto 0)
        );
    end component;

    component incrementer_4bit
        port
        (
            i_A: in std_logic_vector (3 downto 0);
            o_R: out std_logic_vector (3 downto 0);
            o_CARRY_OUT: out std_logic
        );
    end component;

    component adder_4bit is
        port
        (
```



```

        i_A: in std_logic_vector (3 downto 0);
        i_B: in std_logic_vector (3 downto 0);
        o_R: out std_logic_vector (3 downto 0);
        o_CARRY_OUT: out std_logic

    );
end component;

signal w_INVERTED_B: std_logic_vector (3 downto 0);
signal w_COMPLEMENT_B: std_logic_vector (3 downto 0);

begin
    u_INVERTER: inverter_4bit port map(i_B, w_INVERTED_B);
    u_INCREMENTER: incrementer_4bit port map(w_INVERTED_B,
w_COMPLEMENT_B, open);
    u_ADDER: adder_4bit port map(i_A, w_COMPLEMENT_B, o_R,
o_CARRY_OUT);
end arch_subtractor_4bit;

```

Fonte: O autor.

Figura 32 – Testbench do subtrator de quatro bits.

```

library IEEE;
use IEEE.std_logic_1164.all;

entity subtractor_4bit_tb is
end subtractor_4bit_tb;

architecture arch_subtractor_4bit_tb of subtractor_4bit_tb is

    component subtractor_4bit is
        port
        (
            i_A: in std_logic_vector (3 downto 0);
            i_B: in std_logic_vector (3 downto 0);
            o_R: out std_logic_vector (3 downto 0);
            o_CARRY_OUT: out std_logic
        );
    end component;

    signal w_A: std_logic_vector (3 downto 0);
    signal w_B: std_logic_vector (3 downto 0);
    signal w_R: std_logic_vector (3 downto 0);
    signal w_CARRY_OUT: std_logic;

begin
    u_SUBTRACTOR: subtractor_4bit port map (w_A, w_B, w_R, w_CARRY_OUT);

    process
    begin
        -- Test 1
        w_A <= "1001";
        w_B <= "0011";

        wait for 1ns;
        assert (w_R = "0101") report "Fail subtractor_4bit -> Test 1" severity error;
    end process;
end arch_subtractor_4bit_tb;

```

```

assert (w_CARRY_OUT = '0') report "Fail subtractor_4bit -> Test 1" severity error;

-- Test 2
w_A <= "1111";
w_B <= "0001";

wait for 1ns;
assert (w_R = "1110") report "Fail subtractor_4bit -> Test 2" severity error;
assert (w_CARRY_OUT = '1') report "Fail subtractor_4bit -> Test 2" severity error;

-- Test 3
w_A <= "0000";
w_B <= "0001";

wait for 1ns;
assert (w_R = "1111") report "Fail subtractor_4bit -> Test 3" severity error;
assert (w_CARRY_OUT = '0') report "Fail subtractor_4bit -> Test 3" severity error;

wait;
end process;
end arch_subtractor_4bit_tb;

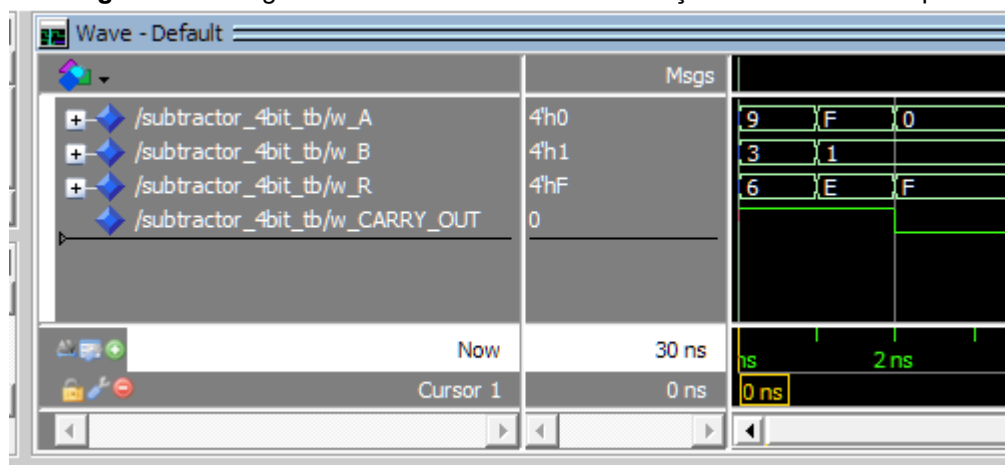
```

Fonte: O autor.

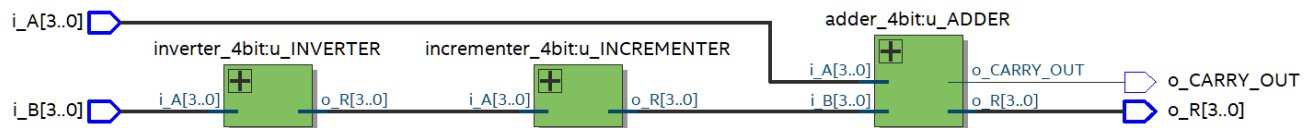
Através do diagrama de forma de onda é possível confirmar que o circuito se comporta da maneira esperada:

- No instante 0ns, o valor 9 (1001) e 3 (0011) são testados, produzindo o valor 6 (0110) como resultado;
- No instante 0ns, o valor F (1111) e 1 (0001) são testados, produzindo o valor E (1110) como resultado;
- No instante 0ns, o valor 0 (0000) e 1 (0001) são testados, produzindo o valor F (1111) como resultado, que em complemento de 2 é igual à -1;

Figura 33 – Diagrama de forma de onda da simulação do subtrator de quatro bits.



Fonte: O autor.

Figura 34 – Diagrama RTL do subtrator de quatro bits.

Fonte: O autor.

2.10. REGISTRADOR DE QUATRO BITS

O registrador de quatro bits tem como entrada um valor de quatro bits, um bit que indica se o valor deve ser salvo ou não (LOAD), e o relógio (CLK). O valor de entrada é atribuído para a saída quando o clock tem uma borda de subida e LOAD é igual a 1.

Figura 35 – Entidade e arquitetura do registrador de quatro bits.

```

library IEEE;
use IEEE.std_logic_1164.all;

entity register_4bit is
    port
    (
        i_A: in std_logic_vector (3 downto 0);
        i_LOAD: in std_logic;
        i_CLK: in std_logic;
        o_R: out std_logic_vector (3 downto 0)
    );
end register_4bit;

architecture arch_register_4bit of register_4bit is
begin
    process (i_A, i_LOAD, i_CLK)
    begin
        if (rising_edge(i_CLK)) then
            if (i_LOAD = '1') then
                o_R <= i_A;
            end if;
        end if;
    end process;
end arch_register_4bit;

```

Fonte: O autor.

Figura 36 – Testbench do registrador de quatro bits.

```

library IEEE;
use IEEE.std_logic_1164.all;

entity register_4bit_tb is
end register_4bit_tb;

architecture arch_register_4bit_tb of register_4bit_tb is

component register_4bit is
    port
    (
        i_A: in std_logic_vector (3 downto 0);
        i_LOAD: in std_logic;
        i_CLK: in std_logic;
        o_R: out std_logic_vector (3 downto 0)
    );
end component;

signal w_A: std_logic_vector (3 downto 0);
signal w_LOAD: std_logic;
signal w_CLK: std_logic;
signal w_R: std_logic_vector (3 downto 0);

begin
    u_REGISTER: register_4bit port map (w_A, w_LOAD, w_CLK, w_R);

    process
    begin
        -- Test 1
        w_A <= "1001";
        w_LOAD <= '1';

        -- Clock pulse
        w_CLK <= '0';
        wait for 1ns;
        w_CLK <= '1';
        wait for 1ns;
        w_CLK <= '0';

        wait for 1ns;
        assert (w_R = "1001") report "Fail register_4bit -> Test 1" severity error;

        -- Test 2
        w_A <= "1101";
        w_LOAD <= '0';

        -- Clock pulse
        w_CLK <= '0';
        wait for 1ns;
        w_CLK <= '1';
        wait for 1ns;
        w_CLK <= '0';

        wait for 1ns;
        assert (w_R = "1001") report "Fail register_4bit -> Test 2" severity error;

        -- Test 3
        w_LOAD <= '1';

        -- Clock pulse
        w_CLK <= '0';
        wait for 1ns;
        w_CLK <= '1';
    end process;
end arch_register_4bit_tb;

```

```

wait for 1ns;
w_CLK <= '0';

wait for 1ns;
assert (w_R = "1101") report "Fail register_4bit -> Test 3" severity error;

wait;
end process;
end arch_register_4bit_tb;

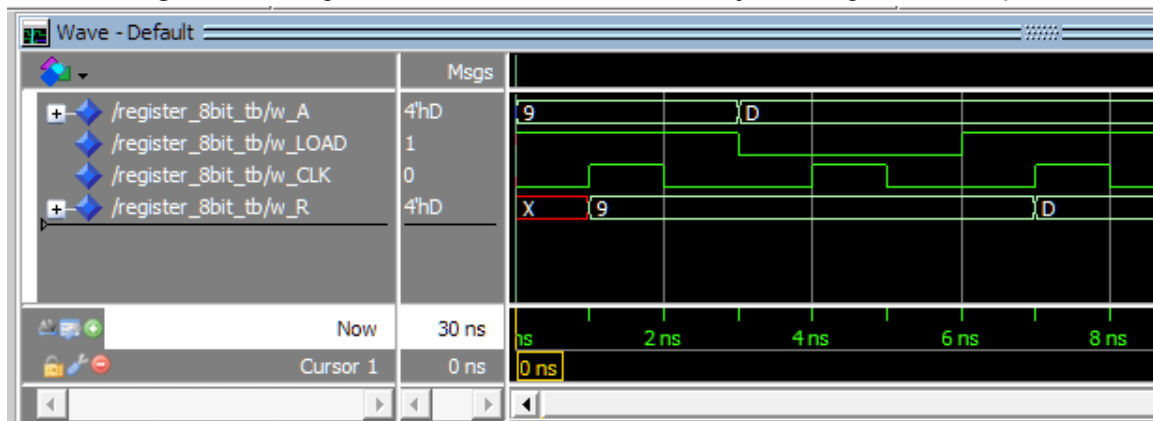
```

Fonte: O autor.

Através do diagrama de forma de onda é possível confirmar que o circuito se comporta da maneira esperada:

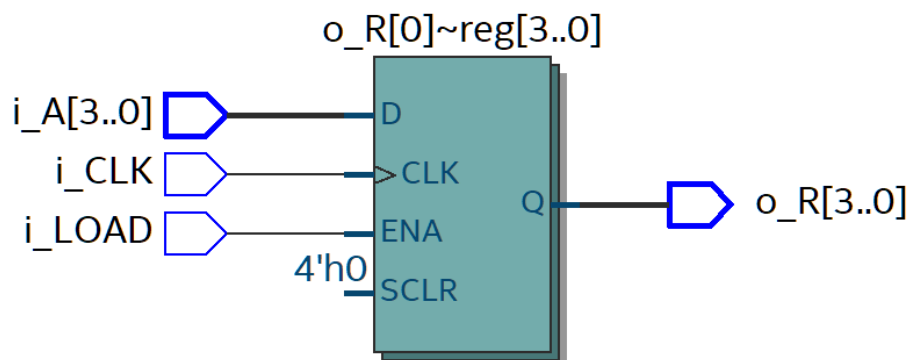
- No instante 0ns, o valor 9 é atribuído à entrada, mas esse valor é atribuído à saída somente no instante 1ns, quando LOAD é igual a 1 e CLK tem uma borda de subida.
- O valor de saída permanece em 9 até o instante 7ns, quando LOAD é igual à 1 e CLK tem uma borda de subida. Nesse instante, a saída recebe o valor de entrada, que é D (1101).

Figura 37 – Diagrama de forma de onda da simulação do registrador de quatro bits.



Fonte: O autor.

Figura 38 – Diagrama RTL do registrador de quatro bits.



Fonte: O autor.

3. PROJETO FINAL DA ULA

A ULA final é composta de duas entradas de quatro bits (A e B), um seletor de três bits (SEL), uma entrada que carrega o registrador (LOAD) e uma entrada de relógio (CLK). A única saída é o resultado da operação selecionada aplicada nas entradas.

Internamente, um multiplexador de oito entradas de quatro bits foi usado para selecionar o resultado correto com base no valor do seletor.

Figura 39 – Entidade e arquitetura da ULA.

```
library IEEE;
use IEEE.std_logic_1164.all;

entity ula_4bit is
    port
    (
        i_A: in std_logic_vector (3 downto 0);
        i_B: in std_logic_vector (3 downto 0);
        i_SEL: in std_logic_vector (2 downto 0);
        i_LOAD: in std_logic;
        i_CLK: in std_logic;
        o_R: out std_logic_vector (3 downto 0)
    );
end ula_4bit;

architecture arch_ula_4bit of ula_4bit is

    component adder_4bit is
        port
        (
            i_A: in std_logic_vector (3 downto 0);
            i_B: in std_logic_vector (3 downto 0);
            o_R: out std_logic_vector (3 downto 0);
            o_CARRY_OUT: out std_logic
        );
    end component;

    component subtractor_4bit is
        port
        (
            i_A: in std_logic_vector (3 downto 0);
            i_B: in std_logic_vector (3 downto 0);
```

```

        o_R: out std_logic_vector (3 downto 0);
        o_CARRY_OUT: out std_logic
    );
end component;

component incrementer_4bit
    port
    (
        i_A: in std_logic_vector (3 downto 0);
        o_R: out std_logic_vector (3 downto 0);
        o_CARRY_OUT: out std_logic
    );
end component;

component and_4bit is
    port
    (
        i_A: in std_logic_vector (3 downto 0);
        i_B: in std_logic_vector (3 downto 0);
        o_R: out std_logic_vector (3 downto 0)
    );
end component;

component or_4bit is
    port
    (
        i_A: in std_logic_vector (3 downto 0);
        i_B: in std_logic_vector (3 downto 0);
        o_R: out std_logic_vector (3 downto 0)
    );
end component;

component xor_4bit is
    port
    (
        i_A: in std_logic_vector (3 downto 0);
        i_B: in std_logic_vector (3 downto 0);
        o_R: out std_logic_vector (3 downto 0)
    );
end component;

component inverter_4bit is
    port

```

```

    (
        i_A: in std_logic_vector (3 downto 0);
        o_R: out std_logic_vector (3 downto 0)
    );
end component;

component mux_8x4bit is
    port
    (
        i_A_0: in std_logic_vector (3 downto 0);
        i_A_1: in std_logic_vector (3 downto 0);
        i_A_2: in std_logic_vector (3 downto 0);
        i_A_3: in std_logic_vector (3 downto 0);
        i_A_4: in std_logic_vector (3 downto 0);
        i_A_5: in std_logic_vector (3 downto 0);
        i_A_6: in std_logic_vector (3 downto 0);
        i_A_7: in std_logic_vector (3 downto 0);
        i_SEL: in std_logic_vector (2 downto 0);
        o_R: out std_logic_vector (3 downto 0)
    );
end component;

component register_4bit is
    port
    (
        i_A: in std_logic_vector (3 downto 0);
        i_LOAD: in std_logic;
        i_CLK: in std_logic;
        o_R: out std_logic_vector (3 downto 0)
    );
end component;

signal w_ADDER_RESULT: std_logic_vector (3 downto 0);
signal w_SUBTRACTOR_RESULT: std_logic_vector (3 downto 0);
signal w_INCREMENTER_RESULT: std_logic_vector (3 downto 0);
signal w_AND_RESULT: std_logic_vector (3 downto 0);
signal w_OR_RESULT: std_logic_vector (3 downto 0);
signal w_XOR_RESULT: std_logic_vector (3 downto 0);
signal w_INVERTER_RESULT: std_logic_vector (3 downto 0);
signal w_MUX_RESULT: std_logic_vector (3 downto 0);

begin
    u_ADDER: adder_4bit port map(i_A, i_B, w_ADDER_RESULT, open);

```



```

        u_SUBTRACTOR: subtractor_4bit port map(i_A, i_B,
w_SUBTRACTOR_RESULT, open);
        u_INCREMENTER: incrementer_4bit port map(i_A,
w_INCREMENTER_RESULT, open);
        u_AND: and_4bit port map(i_A, i_B, w_AND_RESULT);
        u_OR: or_4bit port map(i_A, i_B, w_OR_RESULT);
        u_XOR: xor_4bit port map(i_A, i_B, w_XOR_RESULT);
        u_INVERTER: inverter_4bit port map(i_A, w_INVERTER_RESULT);

        u_MUX: mux_8x4bit port map (
            w_ADDER_RESULT,
            w_SUBTRACTOR_RESULT,
            w_INCREMENTER_RESULT,
            i_A,
            w_AND_RESULT,
            w_OR_RESULT,
            w_XOR_RESULT,
            w_INVERTER_RESULT,
            i_SEL,
            w_MUX_RESULT
        );

        u_REGISTER: register_4bit port map (
            w_MUX_RESULT,
            i_LOAD,
            i_CLK,
            O_R
        );
end arch_ula_4bit;

```

Fonte: O autor.

Figura 40 – Testbench da ULA.

```

library IEEE;
use IEEE.std_logic_1164.all;

entity ula_4bit_tb is
end ula_4bit_tb;

architecture arch_ula_4bit_tb of ula_4bit_tb is

component ula_4bit is
    port
    (
        i_A: in std_logic_vector (3 downto 0);
        i_B: in std_logic_vector (3 downto 0);
        i_SEL: in std_logic_vector (2 downto 0);
        i_LOAD: in std_logic;
        i_CLK: in std_logic;

```

```

        o_R: out std_logic_vector (3 downto 0)
    );
end component;

signal w_A: std_logic_vector (3 downto 0);
signal w_B: std_logic_vector (3 downto 0);
signal w_SEL: std_logic_vector (2 downto 0);
signal w_LOAD: std_logic;
signal w_CLK: std_logic;
signal w_R: std_logic_vector (3 downto 0);

begin
    u_ULA: ula_4bit port map (w_A, w_B, w_SEL, w_LOAD, w_CLK, w_R);

    process
    begin
        w_LOAD <= '1';

        -- Test 1 - ADD
        w_A <= "1001";
        w_B <= "0011";
        w_SEL <= "000";

        -- Clock pulse
        w_CLK <= '0';
        wait for 1ns;
        w_CLK <= '1';
        wait for 1ns;
        w_CLK <= '0';

        wait for 1ns;
        assert (w_R = "1100") report "Fail ula_4bit -> Test 1" severity error;

        -- Test 2 - SUBTRACT
        w_A <= "1001";
        w_B <= "0011";
        w_SEL <= "001";

        -- Clock pulse
        w_CLK <= '0';
        wait for 1ns;
        w_CLK <= '1';
        wait for 1ns;
        w_CLK <= '0';

        wait for 1ns;
        assert (w_R = "0110") report "Fail ula_4bit -> Test 2" severity error;

        -- Test 3 - INCREMENT
        w_A <= "1101";
        w_SEL <= "010";

        -- Clock pulse
        w_CLK <= '0';
        wait for 1ns;
        w_CLK <= '1';
        wait for 1ns;
        w_CLK <= '0';

        wait for 1ns;
        assert (w_R = "1110") report "Fail ula_4bit -> Test 3" severity error;

        -- Test 4 - A
        w_A <= "0011";

```

```

w_SEL <= "011";

    -- Clock pulse
    w_CLK <= '0';
    wait for 1ns;
    w_CLK <= '1';
    wait for 1ns;
    w_CLK <= '0';

    wait for 1ns;
    assert (w_R = "0011") report "Fail ula_4bit -> Test 4" severity error;

-- Test 5 - AND
w_A <= "1101";
w_B <= "1110";
w_SEL <= "100";

    -- Clock pulse
    w_CLK <= '0';
    wait for 1ns;
    w_CLK <= '1';
    wait for 1ns;
    w_CLK <= '0';

    wait for 1ns;
    assert (w_R = "1100") report "Fail ula_4bit -> Test 5" severity error;

-- Test 6 - OR
w_A <= "1001";
w_B <= "0011";
w_SEL <= "101";

    -- Clock pulse
    w_CLK <= '0';
    wait for 1ns;
    w_CLK <= '1';
    wait for 1ns;
    w_CLK <= '0';

    wait for 1ns;
    assert (w_R = "1011") report "Fail ula_4bit -> Test 6" severity error;

-- Test 7 - XOR
w_A <= "1001";
w_B <= "1011";
w_SEL <= "110";

    -- Clock pulse
    w_CLK <= '0';
    wait for 1ns;
    w_CLK <= '1';
    wait for 1ns;
    w_CLK <= '0';

    wait for 1ns;
    assert (w_R = "0010") report "Fail ula_4bit -> Test 7" severity error;

-- Test 8 - INVERTER
w_A <= "1000";
w_SEL <= "111";

    -- Clock pulse
    w_CLK <= '0';
    wait for 1ns;

```

```

        w_CLK <= '1';
        wait for 1ns;
        w_CLK <= '0';

        wait for 1ns;
        assert (w_R = "0111") report "Fail ula_4bit -> Test 8" severity error;

    wait;
end process;
end arch_ula_4bit_tb;

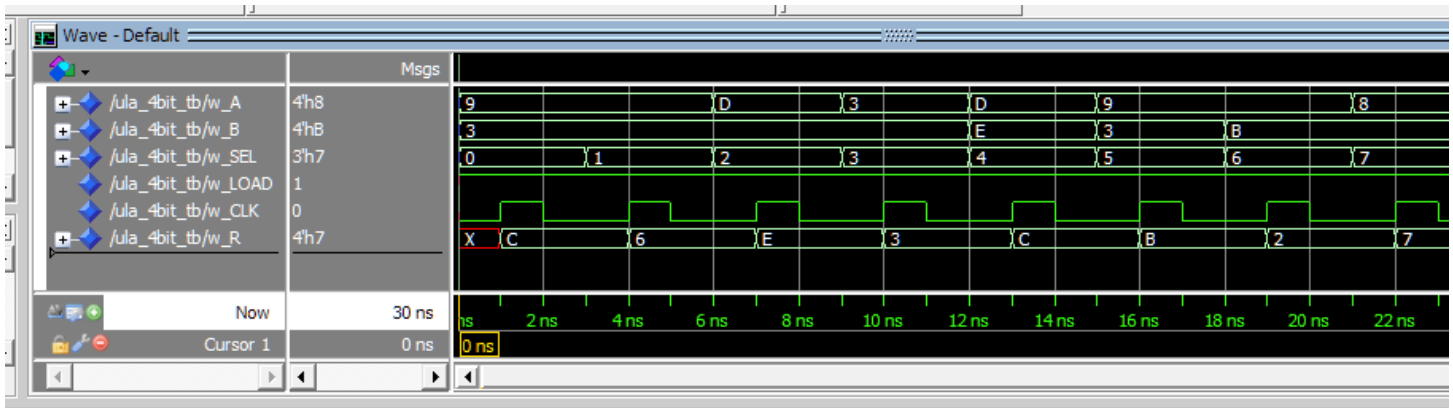
```

Fonte: O autor.

Através do diagrama de forma de onda é possível confirmar que o circuito se comporta da maneira esperada:

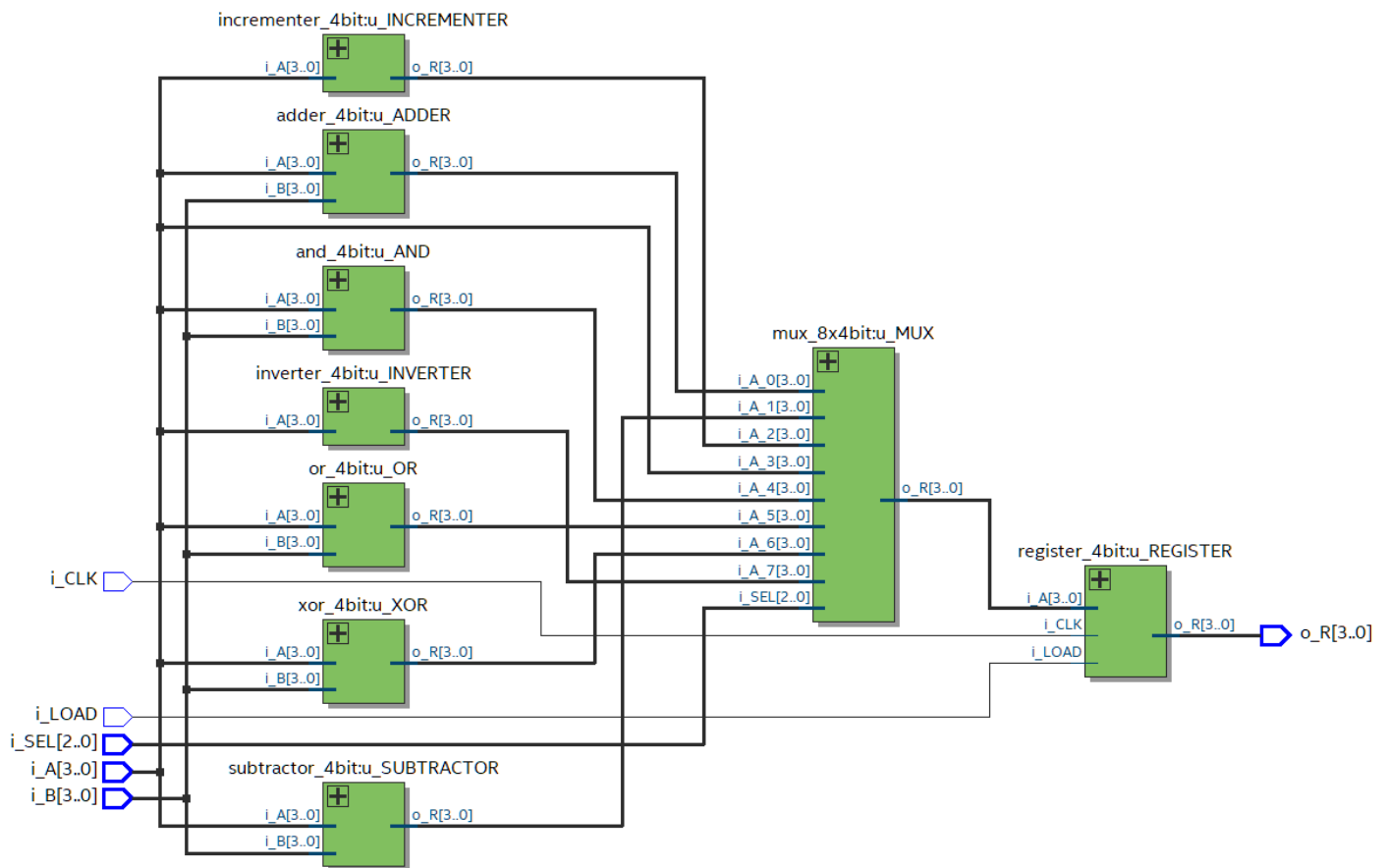
- Quando ocorre uma borda de subida no relógio no instante 1ns, o resultado é C (12), que é a soma dos dois valores de entrada (9 e 3) já que SEL é 000 (operação de soma).
- Quando ocorre uma borda de subida no relógio no instante 4ns, o resultado é 6, que é a subtração dos dois valores de entrada (9 e 3) já que SEL é 001 (operação de subtração).
- Quando ocorre uma borda de subida no relógio no instante 7ns, o resultado é E (14), que é o primeiro valor de entrada (D) mais um, já que SEL é 010 (incremento).
- Quando ocorre uma borda de subida no relógio no instante 10ns, o resultado é 3, que é o mesmo valor da primeira entrada, já que SEL é 011 (saída recebe o valor de A);
- Quando ocorre uma borda de subida no relógio no instante 13ns, o resultado é C (12 - 1100), que é o valor de A (1101) AND B (1110), já que SEL é 100 (operação AND);
- Quando ocorre uma borda de subida no relógio no instante 16ns, o resultado é B (11 - 1011), que é o valor de A (1001) OR B (0011), já que SEL é 101 (operação OR);
- Quando ocorre uma borda de subida no relógio no instante 19ns, o resultado é 2 (0010), que é o valor de A (1001) XOR B (1011), já que SEL é 110 (operação XOR);
- Quando ocorre uma borda de subida no relógio no instante 22ns, o resultado é 7 (0111), que é o valor de A (1000) negado, já que SEL é 111 (operação NOT);

Figura 41 – Diagrama de forma de onda da simulação da ULA.



Fonte: O autor.

Figura 42 – Diagrama RTL da ULA.



Fonte: O autor.

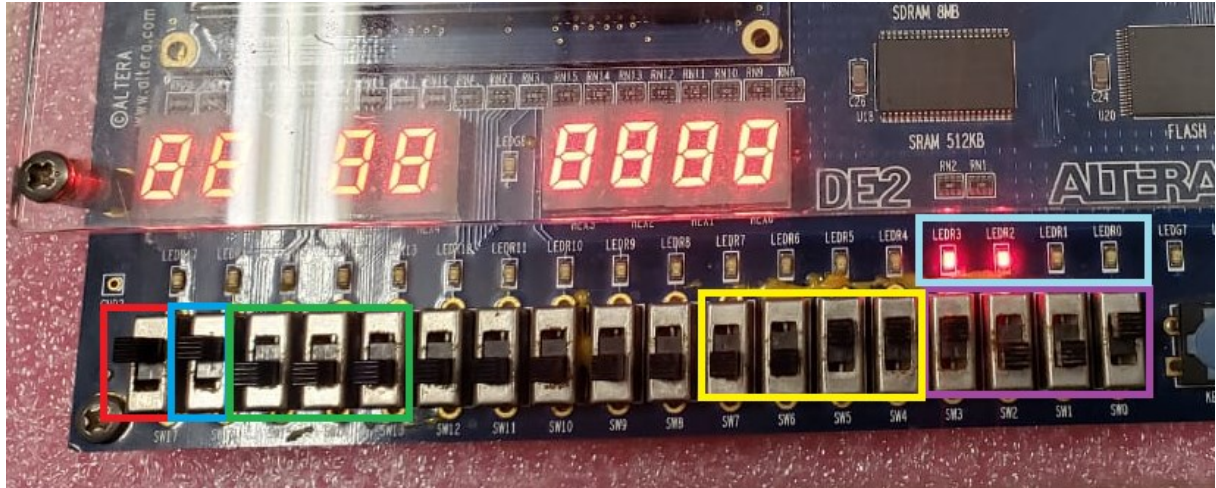
3.1. PROTOTIPAÇÃO EM FPGA

Após o desenvolvimento e síntese da ULA, o circuito gerado foi prototipado no FPGA EP4CE115F29N da família Cyclone II da Altera.

3.1.1. MAPEAMENTO DOS PINOS

O FPGA usado possui 17 *switches* e 25 LEDs no total, mas somente 13 *switches* e 4 LEDs foram usados.

Figura 43 – Mapeamento dos pinos no FPGA.



Fonte: O autor.

Figura 44 – Legenda do mapeamento dos pinos no FPGA.

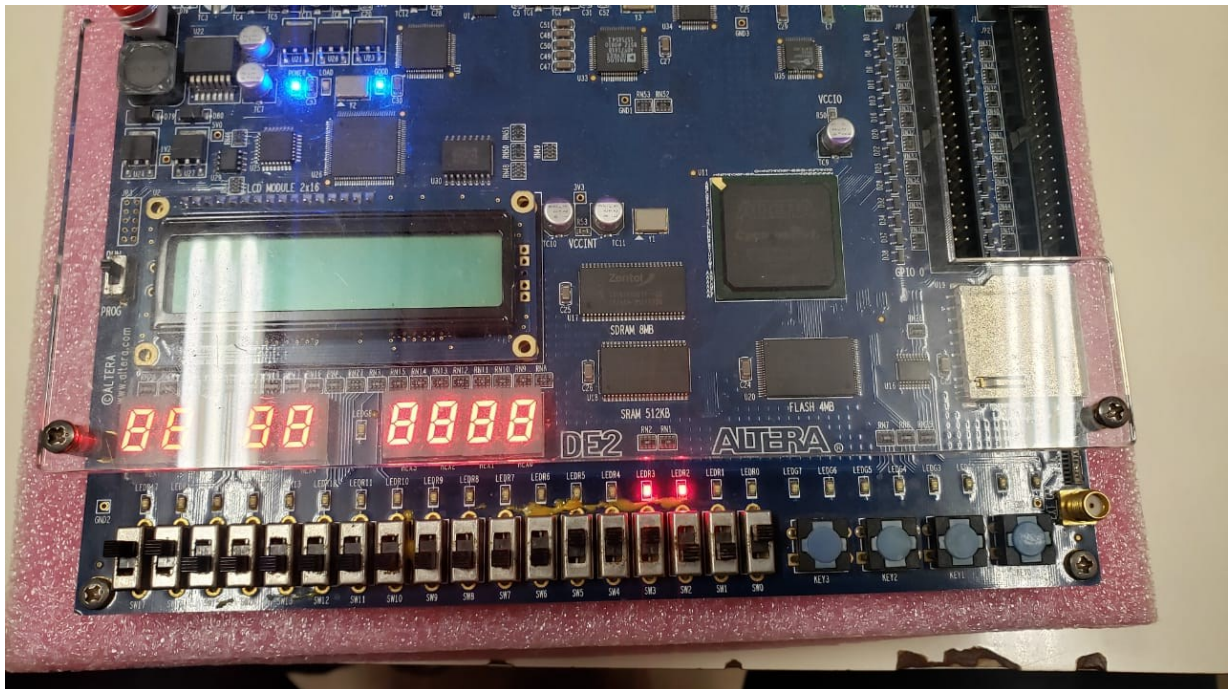
Cor	Significado
Vermelho	Relógio (i_CLK)
Azul	Load (i_LOAD)
Verde	Bits do seletor (i_SEL[2..0])
Amarelo	Bits do valor B (i_B[3..0])
Roxo	Bits do valor A (i_A[3..0])
Azul claro	Saída da ULA (o_R[3..0])

Fonte: O autor.

3.1.2. TESTES NO FPGA

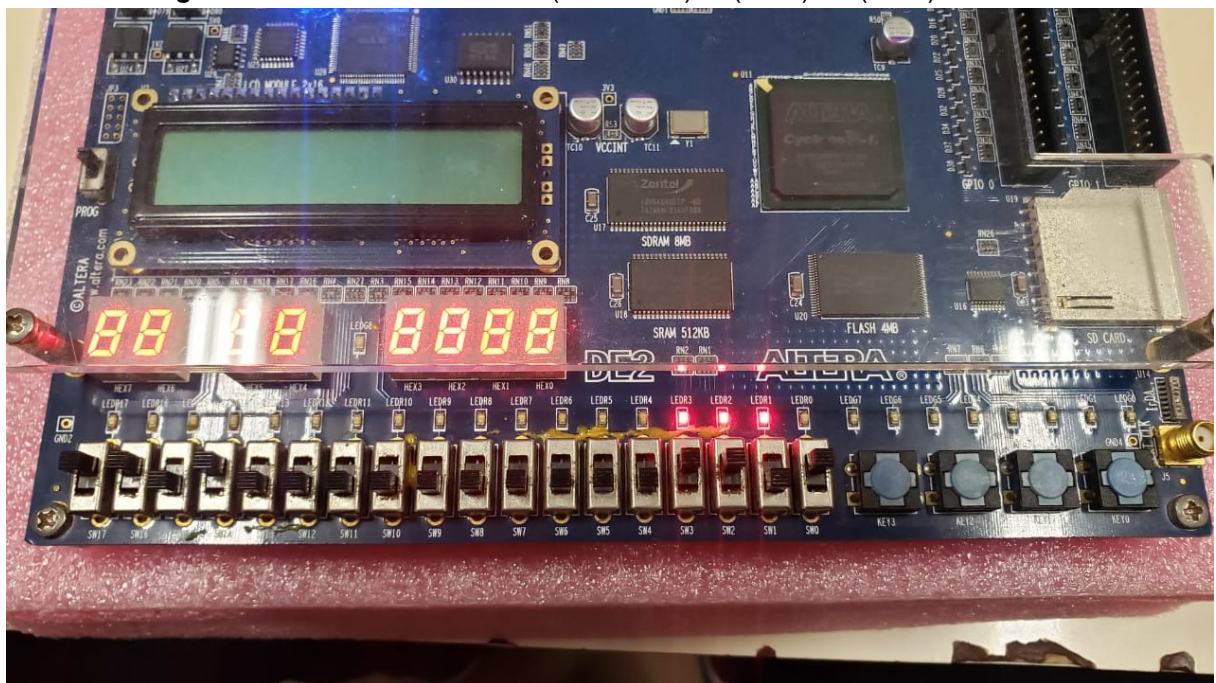
Todos os testes presentes na arquitetura do *testbench* da ULA (*arch_ula_4bit_tb*) foram executados manualmente para verificar a corretude do comportamento do circuito. Todos os resultados obtidos foram idênticos aos resultados das simulações de forma de onda. A foto do segundo teste (subtração) não foi incluída pois ficou muito borrada.

Figura 45 – Teste de soma (SEL é 000). A (1001) + B (0011) resultam em 1100.



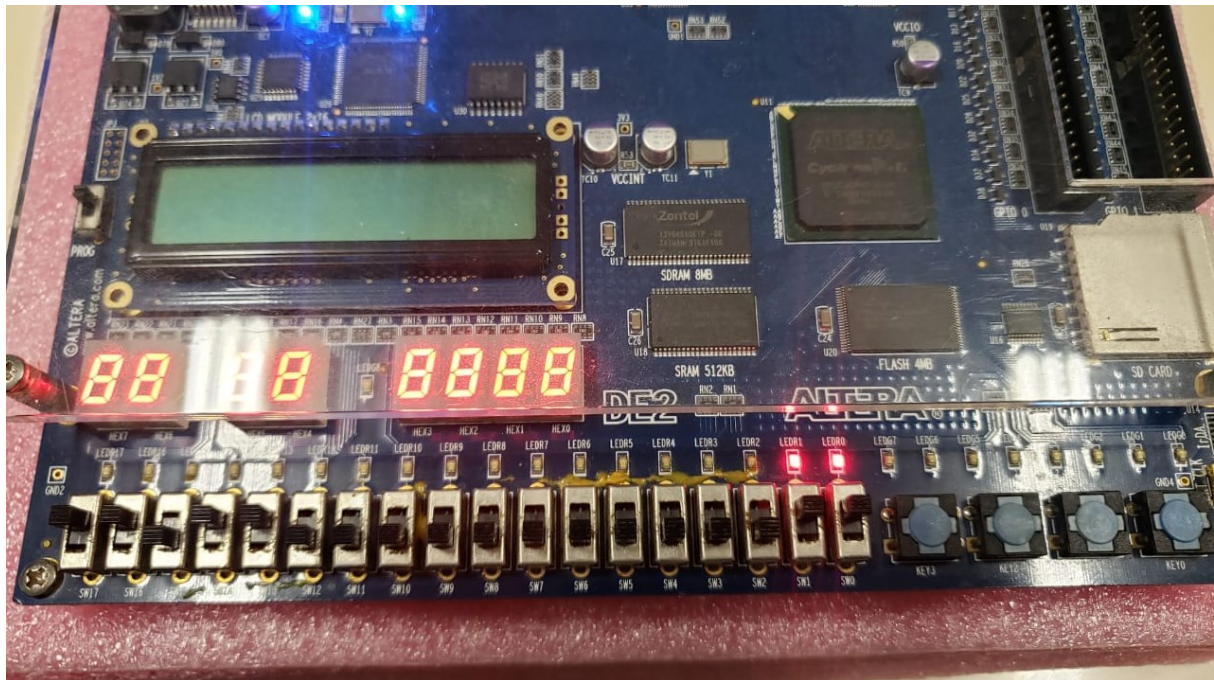
Fonte: O autor.

Figura 46 – Teste de incremento (SEL é 010). A (1101) - B (0000) resultam em 1110.



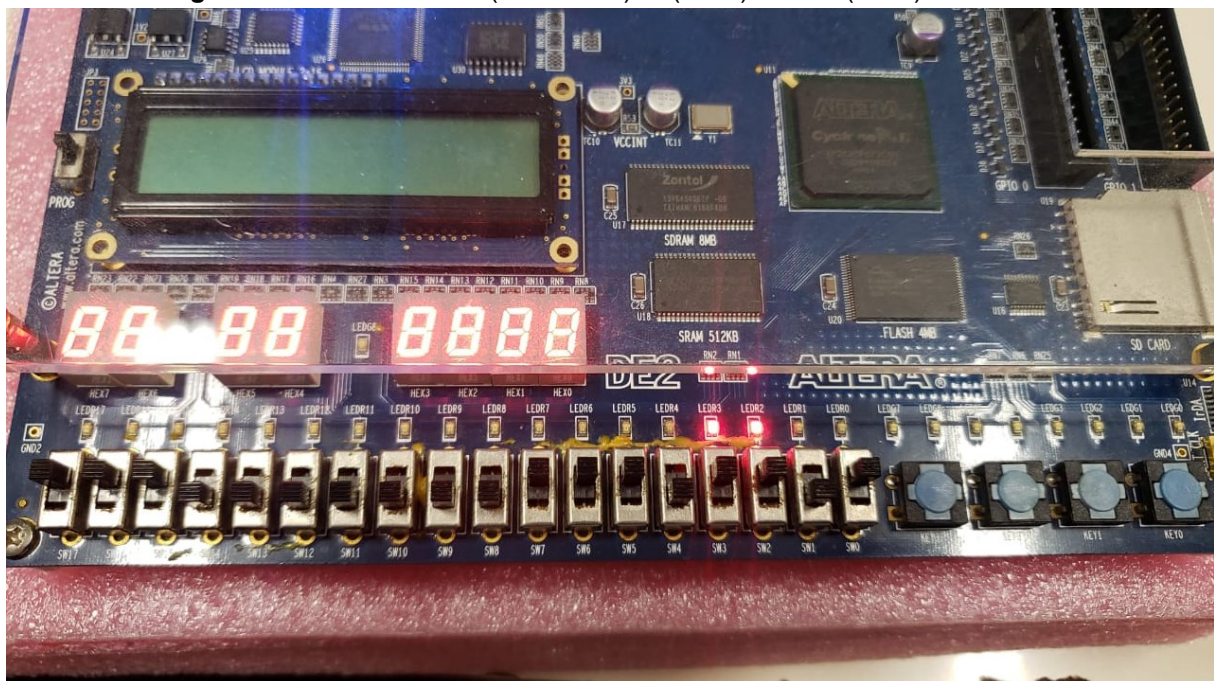
Fonte: O autor.

Figura 47 – Teste do resultado igual ao valor de A (SEL é 011). A (0011) resulta em 0011.



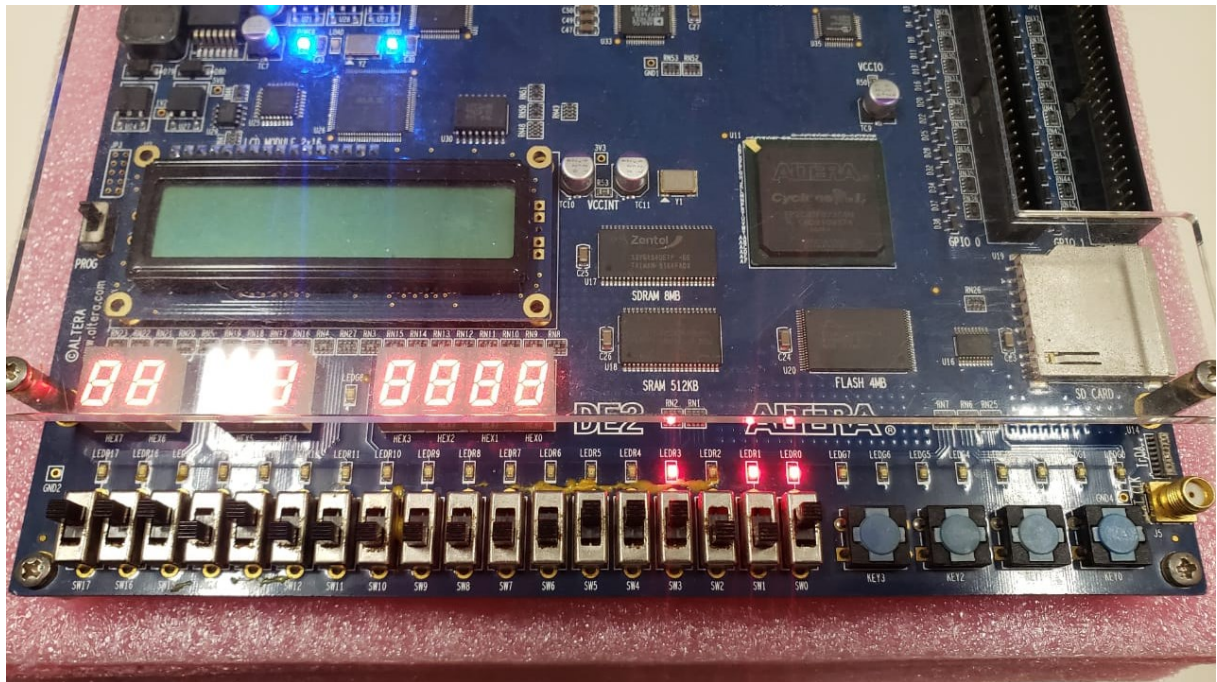
Fonte: O autor.

Figura 48 – Teste de AND (SEL é 100). A (1101) AND B (1110) resultam em 1100.



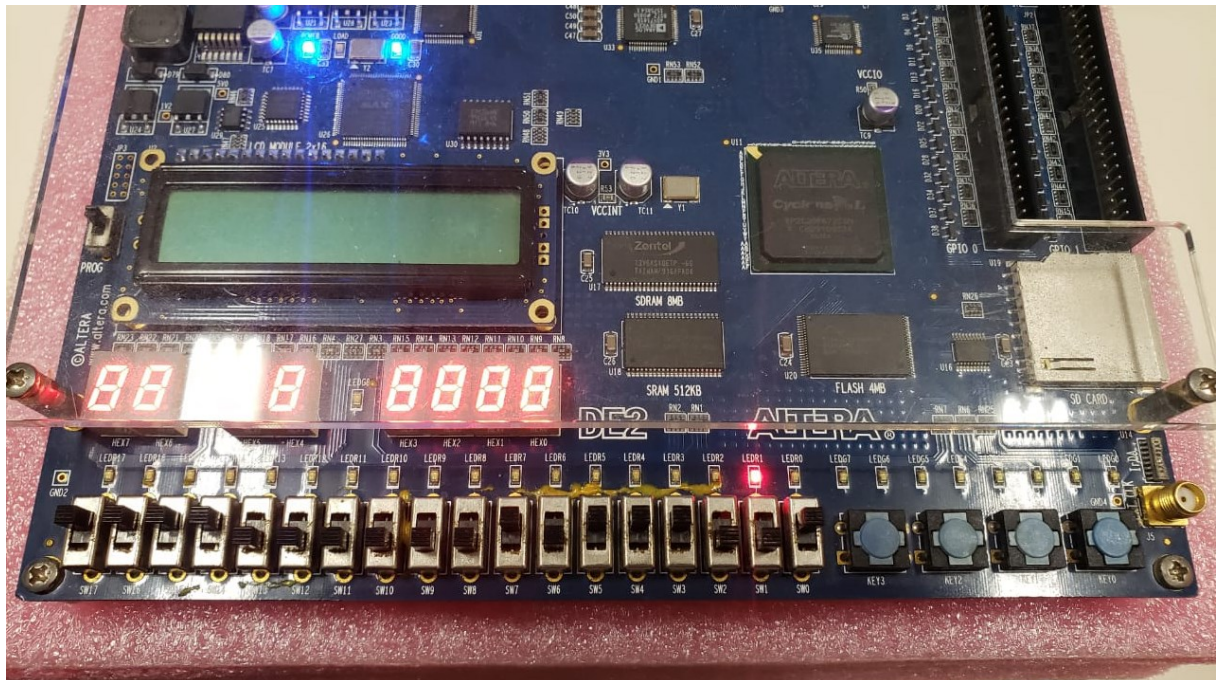
Fonte: O autor.

Figura 49 – Teste de OR (SEL é 101). A (1001) AND B (0011) resultam em 1011.



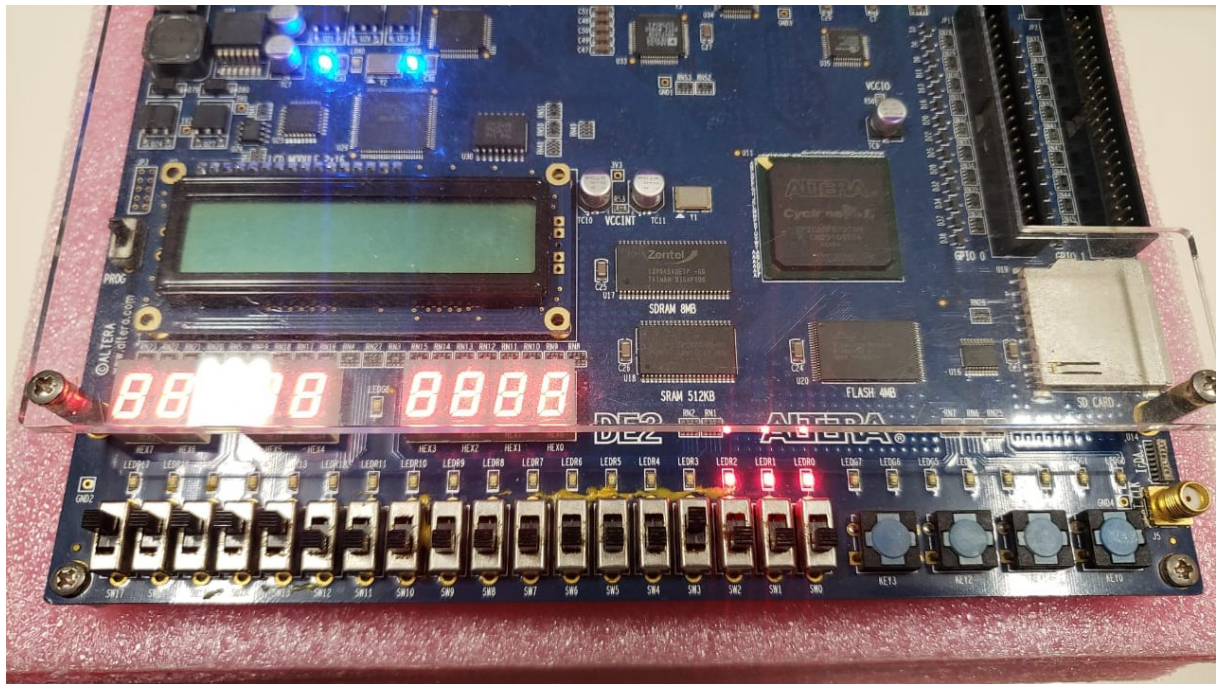
Fonte: O autor.

Figura 50 – Teste de XOR (SEL é 110). A (1001) AND B (1011) resultam em 0010.



Fonte: O autor.

Figura 51 – Teste de NOT (SEL é 111). A (1000) resulta em 0111.



Fonte: O autor.